Boogle Dox — Architecture & Engineering Decisions

This describes the code that exists now. Collaborative Drafts are not built. My original proposed architecture is archived in `docs/archive/chatgpt-planning-pack/ARCHITECTURE.md`. Most of it held up; the differences are noted below.

Why this stack

I chose a JavaScript-based web stack because Boogle Dox is designed to run directly in a browser. JavaScript is suited to interactive editor controls, formatting, menus, and immediate feedback. I have worked with React and Node.js before, so using a familiar ecosystem let me spend the timebox on product behavior rather than learning a new framework. The code is **TypeScript**, not plain JavaScript: static types help catch mismatched document and permission data during development.

Layer	Planned	Built
Frontend	Next.js/React, TypeScript, Tailwind	Next.js 16.4.5 App Router, React 19.3.0, TypeScript, Tailwind CSS 4
Editor	Tiptap, JSON content	Tiptap 3.32.2, stored as Tiptap JSON
Backend	Next.js server routes	Route handlers, plus `src/proxy.ts` (the Next.js 16 proxy) for sessions
Persistence	Hosted Supabase Postgres	Supabase Postgres in production; **PGlite locally** (see below)
Auth	Supabase Auth or seeded demo accounts	Both: Supabase Auth in production, three seeded demo accounts locally
Deployment	Vercel	Vercel, deployed from GitHub `main`
Testing	A focused permissions test	19 Vitest tests, including the hosted RLS SQL

PGlite for local development. Using local PGlite was an implementation suggestion from the coding assistant during the first milestone, which I accepted. It let create, save, and reopen work before any hosted account existed. It runs real Postgres inside Node, uses the same SQL migration files Supabase receives, and lets tests run against a real database in a temporary folder.

Supabase and Vercel in production. These were in my original plan. Supabase provides a hosted Postgres with built-in Auth and row-level security, so permissions are enforced in the database itself. Vercel is the native host for Next.js and deploys automatically from GitHub. Both free tiers cover evaluation, so reviewers don't pay for anything.

Data model as built. It is close to my plan: `profiles` (my planned `users`), `documents` (with `revision`), and `document_permissions` (my planned `document_access`, without `granted_by`). The planned `edit_sessions` table was not built, because the collaboration plan moved to Collaborative Drafts.

Tradeoffs I kept from the plan:

- A rich-text library instead of hand-written contentEditable logic, for reliability and speed.
- Structured JSON instead of raw HTML, so formatting is preserved and rendered safely.
- Roles checked on the server instead of only in the UI, for security and testability.
- A simple page visual and ruler instead of a pagination engine, for familiarity without scope explosion.
- No CRDT/OT engine for simultaneous editing in this timebox.

Why testing, security, and reliability come first for me: I would rather submit a failed test with an understood defect than an artificially passing suite. Permissions are the part of a sharing product that cannot be "mostly right." That standard is written down in `CLAUDE.md`, and the verification evidence is in `DEVELOPMENT_LOG.md`.

Collaborative Drafts (planned, not built)

The next major feature replaces my earlier edit-lease idea:

- Each collaborator works in an independent personal draft.
- Changes come back through a GitHub-style review.
- The team votes on what to merge.

The likely building blocks are a `drafts` table holding a copy of the document JSON plus its base revision, a review record, and votes. The existing revision numbers already give each draft a clear base to compare against. None of this is implemented, and the merge and voting rules are not designed yet. Local mode is verified. Hosted mode is verified against a live Supabase project for the owner account (sign-in, create, save, import, share). Editor and viewer accounts and a public deployment are not yet verified live.

Application

Next.js App Router with React and TypeScript.

- `/` lists documents the current account owns or can access, creates a document, and imports `.txt` or `.md`.
- `/documents/[id]` opens the Tiptap editor. Viewers get a read-only page. Owners can open the share panel.
- `/auth/sign-in` is the hosted sign-in form (email and password).
- `/api/documents` creates and lists documents.
- `/api/documents/[id]` reads and updates one document.
- `/api/documents/import` creates a document from an uploaded text or Markdown file.
- `/api/documents/[id]/access` lists, grants, changes, and removes editor or viewer access.
- `/api/session` reports the mode and current account, and sets the demo account cookie in local mode only.
- `/api/auth/sign-in` and `/api/auth/sign-out` call Supabase Auth. They return 503 unless the mode is hosted.

The editor stores Tiptap JSON, not HTML. Reloading renders that JSON through the editor schema, so formatting is data rather than injected markup.

Runtime mode

`src/lib/auth/runtime-mode.ts` reads `NODE_ENV`, `NEXT_PUBLIC_SUPABASE_URL`, and `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` on the server:

- **local**: development or test without a valid hosted config. PGlite plus the demo cookie.
- **hosted**: an `https://` project URL and an `sb_publishable_...` key. Supabase Auth and Postgres.
- **misconfigured**: production without a valid hosted config, including an `http://` URL or a legacy `eyJ...` JWT. Every request gets 503. PGlite is never opened and the demo cookie is never read.

The production fallback is deliberately closed: a deployment with missing variables must not quietly serve a local database to the public.

Request flow in hosted mode

`src/proxy.ts` (Next.js 16's replacement for middleware) runs before matched routes. It creates a Supabase server client from the request cookies, refreshes the session, and calls `getClaims()`, which verifies the JWT rather than trusting the stored session. When there is no signed-in user:

- `/api/*` returns 401 JSON, except `/api/auth/*`, which must work before a session exists.
- Other pages redirect to `/auth/sign-in`. `/auth/*` pages stay reachable.

When the proxy replaces the response (401 or redirect), it copies the refreshed session cookies and the Cache-Control, Expires, and Pragma headers, so one user's session cannot be cached for another. The matcher skips Next.js static assets and images.

Route handlers do not rely on the proxy alone. `getCurrentUserId()` in `src/lib/auth/current-user.ts` calls `getClaims()` again and returns 401 without a verified subject.

Persistence

Local development uses PGlite, a PostgreSQL build that runs inside the Node process and writes to disk. Startup applies each SQL file in `supabase/migrations` whose id is missing from `schema_migrations`, skipping `*.hosted.sql`. Demo profile rows are inserted by `seedDemoAccounts` after migration, not by the SQL files. `src/lib/db/client.ts` refuses to open PGlite in hosted or misconfigured mode.

The database lives outside the project directory by default (`~/.boogledox/pglite`) because this folder is inside OneDrive. PGlite is one process-wide singleton, so only one server may open that folder at a time.

In hosted mode, `src/lib/documents/repository.ts` delegates to `src/lib/documents/hosted-repository.ts` before any PGlite code runs. That module uses the user-scoped server client (publishable key plus the user's cookies), never a secret key, so Postgres row-level security applies to every query:

- List, read, and insert go through PostgREST. Insert uses the user id from `getClaims()`.
- Save and sharing call SQL functions: `save_document`, `grant_document_access`, `change_document_access`, `revoke_document_access`.

`supabase/migrations/003_rls.hosted.sql` is the hosted security model:

- RLS is enabled on `documents`, `document_permissions`, and `profiles`. Policies allow SELECT of documents the caller owns or has a permission row for, INSERT only with `owner_id` equal to the caller, revision 1, and a Tiptap `doc` object, SELECT of permission rows that belong to the caller or to documents the caller owns, and

SELECT of profiles by any signed-in user. There are no UPDATE or DELETE policies, so direct updates and deletes through the API match no rows.
- The save and sharing functions are `SECURITY DEFINER` with `search_path = public` and schema-qualified `auth.uid()`. A stranger gets "not found", a viewer gets "read only", a stale revision is rejected, the owner cannot be changed or removed through sharing, and a duplicate share maps from the unique violation.
- A before-update trigger blocks id and owner changes and requires `revision = old + 1`.
- User ids stay `text`, so `auth.uid()::text` matches the local demo ids. There is no foreign key to `auth.users` in the portable migrations.

`004_profile_trigger.hosted.sql` inserts a `profiles` row after a Supabase Auth user is created.

Known hosted gaps: any signed-in user can read `profiles`, so registered emails can be enumerated. `schema_migrations` has no RLS. Supabase's default grants to `anon` and `authenticated` have not been inspected on a live project.

Ownership

Local mode: `LOCAL_DEV_USER_ID` in `src/lib/auth/local-user.ts` is Tracy's demo id and the default when the `boogledox_user` cookie is missing or not one of the three allowlisted accounts. The cookie is HttpOnly and SameSite=Lax. Route handlers read it on the server. Request bodies cannot choose an arbitrary owner. Hosted mode ignores that cookie and uses only the verified Supabase subject.

Mutating requests that send an `Origin` header must match the `Host` header.

`src/lib/permissions/access.ts` decides view, edit, and share rights for local mode. The repository loads `document_permissions` and calls those functions before it reads or writes. A missing or unauthorized document returns the same 404. A viewer gets 403 `read_only` on save. Only the owner can grant, change, or revoke access. The owner is not stored as a permission row. In hosted mode the same rules are enforced by the SQL functions and RLS.

Updates require the revision the editor last saved. The update matches that revision and increments it. A mismatch returns HTTP 409 and the editor keeps the unsaved text on screen.

Save path

The editor debounces changes and sends `PATCH /api/documents/:id` with the title, Tiptap JSON, and revision. The status line shows Saving, Saved, Unsaved changes, or the save error. A failed save does not replace the editor contents.

Tests

- `src/lib/documents/access-and-import.test.ts`: the real repository and import parser on a temporary PGlite folder.
- `src/lib/auth/runtime-mode.test.ts`: mode selection and fail-closed rules.
- `src/lib/permissions/rls.postgres.test.ts`: applies 001, 002, and 003 to a separate temporary PGlite, stubs `auth.uid()`, and runs as the `authenticated` role.

- `src/proxy.test.ts`: the real proxy with hosted variables and no session cookie (network stubbed), plus local pass-through and production 503.

These do not replace a test against a live Supabase project, which has not been run.

Not built yet

A live Supabase project, a public deployment, Insert and Tools menus, extra formatting beyond the first milestone, and collaborative drafts. The local demo switcher is not authentication.